

EXP1

AREA ADDITION, CODE, READONLY

ENTRY

EXPORT __main

__main

MOV r0, #10

MOV r1, #0

MOV r2, #1

loop ADD r1, r1, r2

ADD r2, #1

SUBS r0, #1

BNE loop

STOP

NOP

END

2ND

AREA ADDITION, CODE, READONLY

ENTRY

EXPORT __main

__main

ldr r1, =0x12341234

ldr r2, =0x12341234

add r3, r2, r1

L1 B L1

nop

end

3RD

AREA ADDITION, CODE, READONLY

ENTRY

EXPORT __main

__main

LDR R0, =ARRAY

MOV R1, #0

MOV R4, #0

NEXT CMP R1, #6

BEQ STORE

LDRH R3, [R0], #2

ADC R4, R4, R3

ADD R1, R1, #1

B NEXT

STORE LDR R5, =SUM

STR R4, [R5]

STOP B STOP

ARRAY DCW 0x0001, 0x0002, 0x0003, 0x0004, 0x0005, 0x0006

AREA RESULT, DATA, READWRITE

SUM DCD 0

END

4TH

AREA SQUARE, CODE, READONLY

ENTRY

EXPORT __main

__main ; Mark first instruction to execute

LDR R0, = TABLE1 ; Load start address of Lookup table

LDR R1, = 6 ; Load no whose square is to be find

MOV R1, R1, LSL#0x2 ; Generate address corresponding to square of given no

ADD R0, R0, R1; Load address of element in Lookup table

LDR R3, [R0] ; Get square of given no in R3

NOP ; Lookup table contains Squares of no's from 0 to 10

TABLE1 DCD 0X00000000 ; SQUARE OF 0=0

DCD 0X00000001 ; SQUARE OF 1=1

DCD 0X00000004 ; SQUARE OF 2=4

DCD 0X00000009 ; SQUARE OF 3=9

DCD 0X00000010 ; SQUARE OF 4=16

DCD 0X00000019 ; SQUARE OF 5=25

DCD 0X00000024 ; SQUARE OF 6=36

DCD 0X00000031 ; SQUARE OF 7=49

DCD 0X00000040 ; SQUARE OF 8=64

DCD 0X00000051 ; SQUARE OF 9=81

DCD 0X00000064 ; SQUARE OF 10=100

END ; Mark end of file

5TH

AREA LARGEST, CODE, READONLY

ENTRY

EXPORT __main

__main

MOV R5, #6 ; INITIALISE COUNTER TO 6 (i.e. N=7)

LDR R1, =VALUE1 ; LOAD ADDRESS OF ARRAY

LDR R2, [R1], #4 ; LOAD FIRST ELEMENT

LOOP

LDR R4, [R1], #4 ; LOAD NEXT ELEMENT

CMP R2, R4 ; COMPARE

BHI LOOP1 ; IF R2 > R4, SKIP UPDATE

MOV R2, R4 ; ELSE UPDATE MAX

LOOP1

SUBS R5, R5, #1 ; DECREMENT COUNTER

CMP R5, #0

BNE LOOP

LDR R4, =RESULT ; LOAD RESULT ADDRESS

STR R2, [R4] ; STORE RESULT

NOP

NOP

NOP

VALUE1

DCD 0x44444444

DCD 0x22222222

DCD 0x11111111

DCD 0x33333333

DCD 0xAAAAAAAA

DCD 0x88888888

DCD 0x99999999

AREA DATA2, DATA, READWRITE

RESULT DCD 0x0

END

6TH

AREA ASCENDING , CODE, READONLY

ENTRY ; Mark first instruction to execute

EXPORT __main

__main ; Mark first instruction to execute

MOV R8,#4 ; INITIALISE COUNTER TO 4(i.e. N=4)

LDR R2,=CVALUE ; ADDRESS OF CODE REGION

LDR R3,=DVALUE ; ADDRESS OF DATA REGION

LOOP0

```
LDR R1,[R2],#4 ; LOADING VALUES FROM CODE REGION
STR R1,[R3],#4 ; STORING VALUES TO DATA REGION
SUBS R8,R8,#1 ; DECREMENT COUNTER
CMP R8,#0 ; COMPARE COUNTER TO 0
BNE LOOP0 ; LOOP BACK TILL ARRAY ENDS
```

START1

```
MOV R5,#3 ; INITIALISE COUNTER TO 3(i.e. N=4)
MOV R7,#0 ; FLAG TO DENOTE EXCHANGE HAS OCCURED
LDR R1,=DVALUE ; LOADS THE ADDRESS OF FIRST VALUE
```

LOOP1

```
LDR R2,[R1],#4 ; WORD ALIGN TO ARRAY ELEMENT
LDR R3,[R1] ; LOAD SECOND NUMBER
CMP R2,R3 ; COMPARE NUMBERS
BLT LOOP2 ; IF THE FIRST NUMBER IS < THEN GOTO LOOP2
STR R2,[R1],#-4 ; INTERCHANGE NUMBER R2 & R3
STR R3,[R1] ; INTERCHANGE NUMBER R2 & R3
MOV R7,#1 ; FLAG DENOTING EXCHANGE HAS TAKEN PLACE
ADD R1,#4 ; RESTORE THE PTR
```

LOOP2

```
SUBS R5,R5,#1 ; DECREMENT COUNTER
CMP R5,#0 ; COMPARE COUNTER TO 0
BNE LOOP1 ; LOOP BACK TILL ARRAY ENDS
CMP R7,#0 ; COMPARING FLAG
BNE START1 ; IF FLAG IS NOT ZERO THEN GO TO START1
NOP
NOP
NOP ; ARRAY OF 32 BIT NUMBERS(N=4) IN CODE REGION
```

CVALUE

```
DCD 0X44444444 ;
DCD 0X11111111 ;
```

DCD 0X33333333 ;

DCD 0X22222222 ;

AREA DATA1, DATA, READWRITE ; ARRAY OF 32 BIT NUMBERS IN DATA REGION

DVALUE

DCD 0X00000000 ;

END

7TH

AREA SEARCH1, CODE, READONLY

ENTRY

EXPORT __main

__main

MOV r5,#2

MOV r0,#0 ; zero count

MOV r1,#0 ; one count

LDR r2, =VALUE1

LDR r2, [r2]

SECOND

MOV r3,#32

NEXT

MOVS r2,r2,ROR #1

BLO ZERO

ADD r1,r1,#1

B LAST

ZERO

ADD r0,r0,#1

LAST

SUBS r3,r3,#1

BNE NEXT

LDR r2, =VALUE2

LDR r2, [r2]

SUBS r5,r5,#1

BNE SECOND

STOP

B STOP

VALUE1

DCD 0x1234567A

VALUE2

DCD 0x1234567A

END

8TH